

R.I.D.E. - Rent It, Deliver Everything!

Allgemeine Hinweise

- Alle zu bearbeitenden Klassen finden Sie im Paket `de.uniwue.ride`.
- Das Paket befindet sich im Ordner `src`.
- Die Skelette der Klassen sind bereits gegeben, die dort eingefügten Exceptions sind durch den erwarteten Code zu ersetzen.
- Es ist gestattet, aber nicht nötig, neben den geforderten Klassen weitere Klassen oder zusätzliche Methoden zu implementieren.

Unit-Tests

- Es stehen lokale Unit-Tests zum Überprüfen Ihrer Implementierung zur Verfügung.
- Die Testklassen befinden sich im Ordner `test`.
- Jede Klasse in dieser Aufgabenstellung hat eine passende Testklasse (Name: `"Test<Nr.><Klassenname>"`). Diese können auch einzeln ausgeführt werden.
- Die Tests sind von Anfang an ausführbar.
- Sie haben die Möglichkeit die Tests zu **debuggen** und herauszufinden welche Szenarien getestet werden.

Aufgaben

Sie haben das Startup R.I.D.E gegründet und bieten in Würzburg Leihlastenfahrräder mit Elektromotor an. Dazu benötigen Sie eine Verwaltungssoftware um zu kontrollieren, wo sich welche Räder befinden und um Ihren Kunden einen Überblick über die aktuell zur Verfügung stehenden Räder zu geben. Die Software speichert weiterhin den Ladezustand der Räder und berechnet die Kosten für einzelne Fahrten.

Im Folgenden implementieren Sie eine simple Version dieser Verwaltungssoftware.

Die Klausur ist in diese Teilaufgaben gegliedert:

Nummer	Name	Punkte
1.	Bike	15
2.	Trip	17
3.	BikeSharingManager	58
Summe		90

Das Erreichen von **60** Punkten ist hinreichend, aber nicht zwingend notwendig, um die Prüfung zu bestehen. Nach den jeweiligen Teilaufgaben finden Sie eine (unverbindliche) detaillierte Auflistung der Tests mit zugehöriger Punktzahl.

Viel Erfolg!

Die Klasse Bike (15 Punkte)

Objekte dieser Klasse repräsentieren ein Leihlastenfahrrad und speichern somit die verschiedenen Eigenschaften (wie ID, maximale Last, zusätzliche Sitze und Ladezustand). Weiterhin wird in einem `Bike`-Objekt abgespeichert ob das Fahrrad aktuell ausgeliehen ist oder nicht.

Implementieren Sie diesen Konstruktor

```
public Bike(int id, int maxCargo, int seats)
```

Erstellt ein neues Fahrrad mit der ID `id`, der maximalen Last `maxCargo` (in Kilogramm) und `seats` vielen Sitzen.

Werfen Sie eine `IllegalArgumentException`, wenn `maxCargo` kleiner oder gleich 0 oder `seats` kleiner 0 ist (n 0 Sitze sind erlaubt, da es sich hier um die zusätzlichen Sitze neben dem Fahrer Sitz handelt).

Implementieren Sie diese Methoden:

- `public int getId()`
Gibt die ID des Fahrrads zurück.
- `public int getMaxCargo()`
Gibt die maximale Last des Fahrrads zurück.
- `public int getSeats()`
Gibt die Anzahl an Sitzen zurück.
- `public int getChargeState()`
Gibt den Ladezustand des Fahrrads zurück.
Nach der Objekterstellung soll ein Fahrrad hier zunächst immer 100 zurück geben (steht für 100%). Der Ladezustand kann sich dann aber durch die Methoden `setChargeState` und `endTrip` ändern (siehe unten).
- `public void setChargeState(int chargeState)`
Setzt den Ladezustand des Fahrrads auf `chargeState`.
Werfen Sie eine `IllegalArgumentException`, wenn `chargeState` kleiner 0 oder größer 100 ist.
- `public boolean isInUse()`
Gibt `true` zurück, wenn das Fahrrad gerade ausgeliehen ist und sonst `false`.
Nach der Objekterstellung sollen Fahrräder zunächst immer nicht ausgeliehen sein (also hier `false` zurückgeben). Das kann sich durch die Methoden `startTrip` und `endtrip` dann aber ändern (siehe unten).
- `public String toString()`
Die Stringrepräsentation eines Fahrrads soll so aussehen:

```
Fahrrad <id> (Maximale Last: <maxCargo>kg, Anzahl Sitze: <seats>, Batterie: <chargeStat  
e>%)
```

Hinweis:

In den folgenden Methoden (und Klassen) werden als Parameter immer wieder sogenannte Timestamps übergeben. Das sind immer Integer, deren Zahlenwerte wie Zeitangaben oder Uhrzeiten zu verstehen sind. In unserer Aufgabe stehen die Zahlenwerte dabei für *Anzahl der Minuten seit Zeitpunkt XY*.

Das bedeutet z.B., dass wenn ein Timestamp `x` größer als der Timestamp `y` ist, `x` auch später als `y` ist.

- `void startTrip(int now)`
Startet einen Leihvorgang auf dem Fahrrad. Das Fahrrad ist also nach Aufruf dieser Methode ausgeliehen und soll sich auch so in den anderen Methoden dieser Klasse verhalten.
`now` ist die aktuelle Uhrzeit und muss für die folgenden Methoden zwischengespeichert werden.

Werfen Sie eine `IllegalArgumentException` wenn `now` kein valide Timestamp ist (also kleiner als 0 ist).

Werfen Sie eine `IllegalStateException` , wenn das Fahrrad schon ausgeliehen ist.

- `void endTrip(int now)`

Beendet den Leihvorgang auf dem Fahrrad wieder (auch das soll durch die anderen Methoden dieser Klasse erkennbar sein).

`now` ist wieder die aktuelle Uhrzeit.

Weiterhin wird durch diese Methode der Ladezustand des Fahrrads verändert. Für jede ausgeliehene Minute soll die Batterie ein Prozent verlieren. Berechnen Sie also die Zeit, die seit dem Beginn des Leihvorgangs vergangen ist und ziehen Sie das vom aktuellen Ladezustand ab. Sollte der Ladezustand dadurch negativ werden, soll er stattdessen auf 0 gesetzt werden.

Werfen Sie weiterhin eine `IllegalArgumentException` , wenn `now` nicht größer als der Startzeitpunkt des Ausleihvorgangs ist.

Werfen Sie eine `IllegalStateException` , wenn das Fahrrad gerade gar nicht ausgeliehen ist.

- `public int getLastStartTimeStamp()`

Gibt den Startzeitpunkt des aktuellen Ausleihvorgangs zurück.

Sollte das Fahrrad gerade nicht ausgeliehen sein werden Sie eine `IllegalStateException` .

Zur Veranschaulichung der obigen Methoden folgt ein kurzes Beispiel, die Zeilen werden nacheinander ausgeführt:

```
Bike b = new Bike(1,10,2)
b.getChargeState() -> 100
b.isInUse -> false
b.endTrip(10) -> IllegalStateException
b.getLastStartTimeStamp() -> IllegalStateException

b.startTrip(10)
b.isInUse -> true
b.getLastStartTimeStamp() -> 10
b.startTrip(20) -> IllegalStateException
b.endTrip(5) -> IllegalArgumentException

b.endTrip(15)
b.getChargeState() -> 95
b.isinUse -> false
[usw...]
```

Wie Sie in der Klasse sehen, sind die Methoden `equals()` und `hashCode` bereits implementiert und dürfen auch nicht verändert werden! Zwei Fahrräder sind demnach gleich, wenn Sie die gleiche `id` haben.

Test	Punkte
<code>Test01Bike.test01Constructor</code>	1
<code>Test01Bike.test02GetId</code>	0.5
<code>Test01Bike.test03GetMaxCargo</code>	0.5
<code>Test01Bike.test04GetSeats</code>	0.5
<code>Test01Bike.test05GetChargeState</code>	1.5
<code>Test01Bike.test06SetChargeState</code>	1.5

Test	Punkte
Test01Bike.test07IsInUse	1.5
Test01Bike.test08ToString	2
Test01Bike.test09StartAndEndTrip	5
Test01Bike.test10GetLastStartTimeStamp	1
	15

Die Klasse Trip (17 Punkte)

Objekte dieser Klasse repräsentieren einen Leihvorgang oder eine Fahrt und speichern dazu Start- und Endzeitpunkt sowie eine Liste mit den Geschwindigkeiten, die während der Fahrt gefahren wurden.

Implementieren Sie diesen Konstruktor:

```
public Trip(int startTimeStamp, int endTimeStamp, List<Integer> speeds)
```

Erstellt eine neue Fahrt mit dem Startzeitpunkt `startTimeStamp` dem Endzeitpunkt `endTimeStamp` und den Geschwindigkeiten `speeds`.

Wie bereits angesprochen repräsentiert `speeds` die Geschwindigkeiten, die während der Fahrt gefahren wurden. Jedes Element der Liste ist dabei die Geschwindigkeit (in km/h) während einer Sekunde. Im ersten Element steht also die Geschwindigkeit während der ersten Sekunde der Fahrt und im zweiten Element die Geschwindigkeit während der zweiten Sekunde usw.

Werfen Sie eine `IllegalArgumentException` wenn

- die Zeitpunkte kleiner sind als 0
- `endTimeStamp` nicht größer als `startTimeStamp` ist
- `speeds` gleich null ist
- die Anzahl der Elemente in `speeds` nicht exakt dem 60 fachen Abstand von `startTimeStamp` und `endTimeStamp` ist.

Schließlich sind die Zeitpunkte in Minuten angegeben und `speeds` gibt die Geschwindigkeiten (in km/h) auf die Sekunde genau an. Wenn man also zum Beispiel einen `Trip` mit `startTimeStamp=5` und `endTimeStamp=7` erstellt, muss man eine `speeds` Liste mitgeben, die 120 Elemente enthält, da die Fahrt 2 Minuten gedauert hat.

- ein Element aus `speeds` negativ ist.

Implementieren Sie diese Methoden:

- `public int getStartTimeStamp()`
Gibt den Startzeitpunkt der Fahrt zurück.
- `public int getEndTimeStamp()`
Gibt den Endzeitpunkt der Fahrt zurück.
- `public List<Integer> getSpeeds()`
Gibt die Geschwindigkeitsliste der Fahrt zurück.
- `public int getMaxSpeed()`
Gibt das größte Element aus `speeds` zurück.
- `public int getMinSpeed()`
Gibt das kleinste Element aus `speeds` zurück.

- `public double getAverageSpeed()`
Gibt die durchschnittliche Geschwindigkeit (also den Durchschnitt der Elemente in `speeds`) zurück.
- `public int getDuration()`
Gibt die Dauer der Fahrt in Minuten zurück. Diese ergibt sich aus Start- und Endzeitpunkt.
- `public int calculateCost()`
Berechnet die Kosten der Fahrt in Cent. Eine Fahrt kostet demnach für jede angebrochene halbe Stunde 1.50€ also 150ct .
Sollte während der Fahrt zu irgendeinem Zeitpunkt 30km/h oder mehr gefahren worden sein, soll zu den Kosten ein Aufpreis von 5€ dazukommen.

Test	Punkte
Test02Trip.test01Constructor	2
Test02Trip.test02GetStartTimeStamp	0.5
Test02Trip.test03GetEndTimeStamp	0.5
Test02Trip.test04GetSpeeds	0.5
Test02Trip.test05GetMaxSpeed	3
Test02Trip.test06GetMinSpeed	3
Test02Trip.test07GetAverageSpeed	3
Test02Trip.test08GetDuration	2
Test02Trip.test09CalculateCost	2.5
	17

Die Klasse BikeSharingManager (58 Punkte)

Objekte dieser Klasse speichern nun eine Menge von Fahrrädern und wo sich diese aktuell befinden. Weiterhin können über den `BikeSharingManager` auch Fahrten auf Fahrrädern gestartet und beendet werden.

Zunächst ein paar allgemeine Hinweise:

Das Enum Location

Dieses ist bereits implementiert (wie Sie sehen) und repräsentiert in unserer Aufgabe einen Ort an dem die Fahrräder ausgeliehen oder abgegeben werden können. Wie Sie sehen gibt es insgesamt 12 verschiedene Orte.

Bitte verändern Sie `Location` **nicht**.

Tests dieser Klasse

Viele Methoden dieser Klasse haben statt eines Testes zwei zugehörige Tests. Das liegt daran, dass die Anforderungen an die Methoden auf verschiedene Tests aufgeteilt wurde, um leichter Teilpunkte zu vergeben. Die Nummerierung der Tests gibt Ihnen dabei Aufschluss darüber in welcher Reihenfolge sie bestanden werden sollten. Wenn Sie also die Aufgabenstellung abarbeiten, sollten sie die Tests nach und nach in der Reihenfolge der Nummerierung bestehen.

Abhängigkeit zwischen den Klassen

Die Tests dieser Klasse sind von der Funktionsfähigkeit der vorherigen Klassen abhängig. Sollten dort noch Fehler sein können Sie trotzdem hier durch die manuelle Korrektur Teilpunkte bekommen (selbst wenn ein Test nicht bestanden ist).

Nun zurück zum `BikeSharingManager`

Dieser speichert immer ab welche Fahrräder aktuell frei sind (also nicht ausgeliehen) und wo sich diese befinden, sowie welche Fahrräder gerade belegt (also ausgeliehen) sind. Aus diesem Grund werden folgende Instanzvariablen empfohlen:

```
Map<Location, Set<Bike>> freeBikes = new HashMap<>();
Set<Bike> takenBikes = new HashSet<>();
```

`freeBikes` speichert in einer Map zu jeder möglichen Location ab welche Fahrräder gerade dort sind. Die Fahrräder aus diesen Mengen sind also zusammengefasst alle freien Fahrräder.

`takenBikes` speichert ab welche Fahrräder aktuell belegt sind.

Sie finden diese Instanzvariablen so schon in der Klasse vor und können Sie auch wie oben benutzen. Wenn Sie die geforderten Methoden aber gerne anders umsetzen wollen, können Sie die Variablen auch anders wählen. Das ist nur eine Empfehlung.

Implementieren Sie diesen Konstruktor:

```
public BikeSharingManager()
```

Erzeugt einen neuen `BikeSharingManager`. Der Konstruktor muss hier lediglich dafür sorgen, dass die Instanzvariablen korrekt auf die folgenden Methoden vorbereitet werden.

Implementieren Sie diese Methoden:

- `public void addBike(Bike bike, Location location)`
Fügt bike am Ort location hinzu.
Das bedeutet, dass bike nun als aktuell freies Fahrrad an Ort location im `BikeSharingManager` abgespeichert werden soll.
Werfen Sie eine `IllegalArgumentException` wenn
 - bike oder location null ist
 - bike aktuell laut eigenem Zustand ausgeliehen ist
 - bike schonmal zu diesem `BikeSharingManager` hinzugefügt wurde (sich also unter allen freien oder belegten Fahrrädern befindet).
- `public Set<Bike> getAllFreeBikes()`
Gibt alle aktuell freien Fahrräder zurück (Fahrräder können nur durch die Methode `startTrip` aus dieser Klasse belegt werden).
- `public Set<Bike> getAllFreeBikesAtLocation(Location location)`
Gibt alle Fahrräder zurück, die aktuell frei sind und sich an dem Ort location befinden.
- `public Set<Bike> getAllTakenBikes()`
Gibt alle aktuell belegten (ausgeliehenen) Fahrräder zurück.
- `public Set<Bike> getAllBikes()`
Gibt alle Fahrräder (frei oder belegt) zurück.
- `public Optional<Location> getLocationOfBike(int id)`
Gibt den Ort des Fahrrads mit der ID id zurück.
Geben Sie ein leeres `Optional` zurück, wenn es kein Fahrrad mit ID id gibt oder das Fahrrad aktuell ausgeliehen ist.

- `public Optional<Bike> getBike(int id)`
Gibt das Fahrrad mit der ID `id` zurück.
Geben Sie ein leeres `Optional` zurück, wenn es kein Fahrrad mit ID `id` gibt.
- `public boolean isInUse(int id)`
Gibt zurück ob das Fahrrad mit der ID `id` aktuell belegt (ausgeliehen) ist.
Geben Sie `false` zurück, wenn es kein Fahrrad mit ID `id` gibt.
- `public void startTrip(int bikeId, int now)`
Startet einen neuen Leihvorgang des Fahrrads mit der ID `id` zum Zeitpunkt `now`. Das bedeutet, dass Sie zunächst das dazugehörige Fahrradobjekt ermitteln müssen um auf diesem selbst `startTrip` aufzurufen.
Weiterhin müssen Sie dafür sorgen, dass die Datenstrukturen des `BikeSharingManager` so verändert werden, dass die obigen Methoden sich anders verhalten. Schließlich ist das Fahrrad mit der ID `id` jetzt belegt und sollte z.B. nicht mehr unter allen freien Fahrrädern zurückgegeben werden.
Sollte es kein Fahrrad mit ID `id` geben oder das Fahrrad aktuell nicht frei sein, werfen Sie eine `IllegalArgumentException`.
- `public Trip endTrip(int bikeId, int now, Location location, List<Integer> speeds)`
Beendet den Leihvorgang des Fahrrads mit der ID `id` zum Zeitpunkt `now`. Weiterhin wird das Fahrrad am Ort `location` abgegeben und während der Fahrt wurden die Geschwindigkeiten `speeds` gefahren. Auch hier gibt es mehrere Dinge zu tun:
 - Beenden Sie die Fahrt auf dem zugehörigen Fahrrad Objekt mit `endTrip`.
 - Setzen Sie den Ladezustand des Fahrrads auf `100`
 - Aktualisieren Sie die Datenstrukturen des `BikeSharingManager` sodass das Fahrrad nun nicht mehr belegt sondern frei am Ort `location` zur Verfügung steht.
 - Geben Sie ein `Trip` Objekt zurück, in dem Start- und Endzeitpunkt sowie `speeds` gespeichert ist.
 - Werfen Sie eine `IllegalArgumentException` wenn es kein Fahrrad mit ID `id` gibt oder das Fahrrad aktuell gar nicht ausgeliehen ist.

Test	Punkte
<code>Test03BikeSharingManager.test01Constructor</code>	1
<code>Test03BikeSharingManager.test02AddBike</code>	2
<code>Test03BikeSharingManager.test03GetAllFreeBikes_AfterAddBike</code>	3
<code>Test03BikeSharingManager.test04GetAllFreeBikesAtLocation_AfterAddBike</code>	3
<code>Test03BikeSharingManager.test05GetAllTakenBikes_AfterAddBike</code>	2
<code>Test03BikeSharingManager.test06GetAllBikes_AfterAddBike</code>	3
<code>Test03BikeSharingManager.test07GetLocationOfBike_AfterAddBike</code>	3
<code>Test03BikeSharingManager.test08GetBike_AfterAddBike</code>	3.5
<code>Test03BikeSharingManager.test09isInUse_AfterAddBike</code>	3.5
<code>Test03BikeSharingManager.test10StartTrip</code>	3
<code>Test03BikeSharingManager.test11EndTrip</code>	5

Test	Punkte
Test03BikeSharingManager.test12GetAllFreeBikes_AfterStartAndEndTrip	3.5
Test03BikeSharingManager.test13GetAllFreeBikesAtLocation_AfterStartAndEndTrip	3.5
Test03BikeSharingManager.test14GetAllTakenBikes_AfterStartAndEndTrip	3.5
Test03BikeSharingManager.test15GetAllBikes_AfterStartAndEndTrip	3.5
Test03BikeSharingManager.test16GetLocationOfBike_AfterStartAndEndTrip	4
Test03BikeSharingManager.test17GetBike_AfterStartAndEndTrip	4
Test03BikeSharingManager.test18IsInUse_AfterStartAndEndTrip	4
	58

Ende der Aufgaben

Unverbindliche Punkteverteilung

Die folgende Verteilung ist nur **vorläufig** und kann für die Bewertung noch angepasst werden.

Test	Punkte
Test01Bike.test01Constructor	1
Test01Bike.test02GetId	0.5
Test01Bike.test03GetMaxCargo	0.5
Test01Bike.test04GetSeats	0.5
Test01Bike.test05GetChargeState	1.5
Test01Bike.test06SetChargeState	1.5
Test01Bike.test07IsInUse	1.5
Test01Bike.test08ToString	2
Test01Bike.test09StartAndEndTrip	5
Test01Bike.test10GetLastStartTimeStamp	1
Test02Trip.test01Constructor	2
Test02Trip.test02GetStartTimeStamp	0.5
Test02Trip.test03GetEndTimeStamp	0.5
Test02Trip.test04GetSpeeds	0.5

Test	Punkte
Test02Trip.test05GetMaxSpeed	3
Test02Trip.test06GetMinSpeed	3
Test02Trip.test07GetAverageSpeed	3
Test02Trip.test08GetDuration	2
Test02Trip.test09CalculateCost	2.5
Test03BikeSharingManager.test01Constructor	1
Test03BikeSharingManager.test02AddBike	2
Test03BikeSharingManager.test03GetAllFreeBikes_AfterAddBike	3
Test03BikeSharingManager.test04GetAllFreeBikesAtLocation_AfterAddBike	3
Test03BikeSharingManager.test05GetAllTakenBikes_AfterAddBike	2
Test03BikeSharingManager.test06GetAllBikes_AfterAddBike	3
Test03BikeSharingManager.test07GetLocationOfBike_AfterAddBike	3
Test03BikeSharingManager.test08GetBike_AfterAddBike	3.5
Test03BikeSharingManager.test09isInUse_AfterAddBike	3.5
Test03BikeSharingManager.test10StartTrip	3
Test03BikeSharingManager.test11EndTrip	5
Test03BikeSharingManager.test12GetAllFreeBikes_AfterStartAndEndTrip	3.5
Test03BikeSharingManager.test13GetAllFreeBikesAtLocation_AfterStartAndEndTrip	3.5
Test03BikeSharingManager.test14GetAllTakenBikes_AfterStartAndEndTrip	3.5
Test03BikeSharingManager.test15GetAllBikes_AfterStartAndEndTrip	3.5
Test03BikeSharingManager.test16GetLocationOfBike_AfterStartAndEndTrip	4
Test03BikeSharingManager.test17GetBike_AfterStartAndEndTrip	4
Test03BikeSharingManager.test18IsInUse_AfterStartAndEndTrip	4
	90